THE NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, LAHORE
CS-4031 Compiler Construction — Fall 2026

Lecture 6

From NFA to a Working Scanner: Subset Construction, DFA Minimization, NFA vs. DFA Trade-offs, and the Design of a Lexical-Analyzer Generator

CLO(s) covered:	CLO 1, CLO 2, CLO 3
Dragon Book [DB]:	§3.7 (Alg. 3.20, Subset Construction), §3.9 (Alg. 3.36, DFA Minimization); §3.2 (Input Buffering), §3.5, §3.8 (Design of a Lexical-Analyzer Generator)
Other references:	[ET] Engineering a Compiler, §2.4–2.5; [AP] Modern Compiler Implementation, §2.5

Lecture Roadmap

1. **Subset construction** (Algorithm 3.20): the mechanical algorithm that turns any NFA into an equivalent DFA, worked all the way through on Lecture **??**'s 14-state NFA for $(a \mid b)^*abb$.
2. **DFA minimization** (Algorithm 3.36): merging behaviorally-identical states to produce the smallest possible DFA for the same language – applied to the very DFA we just built.
3. **NFA vs. DFA, revisited**: same recognizing power (Lecture **??** already proved this constructively), but a real space/time trade-off in practice – worth understanding concretely, not just as a slogan.
4. Zooming out from theory to engineering: **how a lexical-analyzer generator is actually built** – combining many token patterns into one automaton, implementing maximal munch by simulating a DFA, resolving pattern ties by rule order, and buffering the input stream efficiently. This is the generic architecture that *any* scanner-generator tool (Flex included) implements under the hood.

This lecture completes the pipeline Lecture **??**'s big picture (§**??**) promised: regular expression → NFA (done in Lecture **??**) → DFA → minimized DFA → transition table → an actual, running lexer (all done here). By the end of today you will have followed one regular expression all the way from formal notation to working scanner code, and you will understand exactly what a tool like Flex is doing on your behalf when you hand it a `.l` file – the concrete Flex syntax itself is covered separately in the supplementary Flex notes, not here.

6.1 From an NFA to a DFA: The Subset Construction

[DB] Lecture **??** left us with a working NFA, but simulating an NFA directly is awkward: at every step there can be several possible next states, so a naive simulator would have to explore every possible sequence of choices to know whether *any* of them accepts (§**??**'s exam-focus box already flagged this). The **subset construction** sidesteps the problem entirely with one elegant idea: instead of asking "which *one* state am I in?", track the entire *set* of NFA states that are reachable so far, all at once. A set of NFA states is itself just one state of a new automaton – and moving between these sets, one input symbol at a time, turns out to be completely deterministic.

6.1.1 Two Helper Operations

Definition: ε-closure and `move`

Let T be a set of NFA states.

- ε-closure(T) is the set of all NFA states reachable from *any* state in T using **zero or more** ε-transitions alone – "every state you could drift into for free, without reading any input." Every state in T is trivially in its own ε-closure (zero transitions is allowed).
- move(T, a) is the set of NFA states reachable by taking **exactly one** transition labeled $a \in \Sigma$ from *some* state in T – "every state you could be in immediately after consuming one a, before taking any further ε-transitions."

Every DFA state the subset construction builds is, underneath, one of these ε-closure sets – which is exactly why DFA states are conventionally drawn or labeled as *sets* of NFA state numbers throughout this section.

Algorithm 3.20 – Subset Construction

1. **Start state.** Compute $D_0 = \varepsilon\text{-closure}(\{n_0\})$, where n_0 is the NFA's start state. D_0 becomes the DFA's start state, and is added to a worklist of DFA states still needing their outgoing transitions computed.
2. **Process the worklist.** While the worklist is nonempty, remove one unmarked DFA state T from it. For *every* input symbol $a \in \Sigma$:
   (a) Compute $U = \varepsilon\text{-closure}(\text{move}(T, a))$.
   (b) If U is empty, there is no transition on a from T – skip it (this becomes an implicit "error"/dead transition, §6.1.3).
   (c) If U is a *new* set not seen before, add it as a new DFA state and place it on the worklist.
   (d) Either way, record the transition $\delta_{\text{DFA}}(T, a) = U$.
3. **Accepting states.** A DFA state T (itself a set of NFA states) is accepting if and only if T contains *at least one* accepting NFA state – one successful NFA path reaching an accept state is exactly what "the set contains an accepting state" captures.
4. **Termination.** Since the NFA has only finitely many states, it has only finitely many *subsets* of states ($2^{|Q|}$ of them, at most) – so this process, which only ever creates DFA states out of such subsets, is guaranteed to terminate; it cannot generate new DFA states forever.

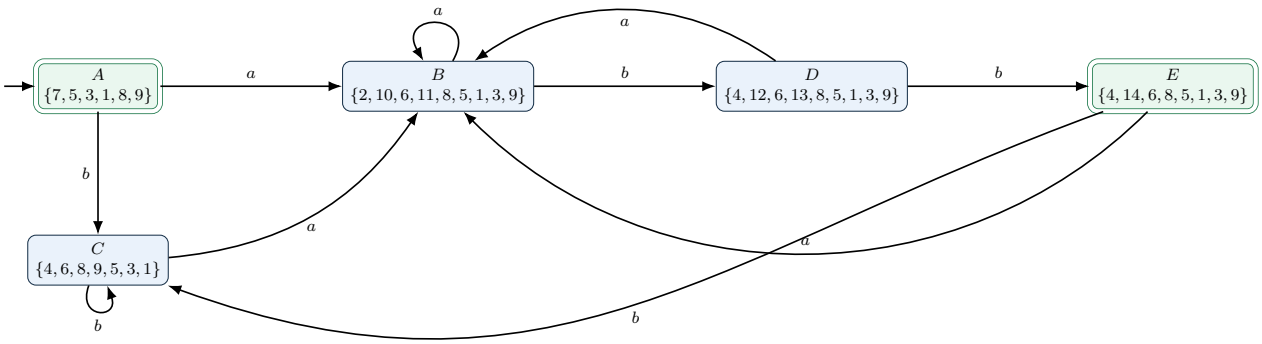
Exam Focus

The single most common hand-execution mistake: forgetting to take the ε-closure *after* `move`, not just at the very start. Every new DFA state is ε-closure(move(T, a)), *not* just move(T, a) – if you skip the closure step, you will silently drop reachable NFA states and build an incorrect DFA that rejects strings it should accept.

6.1.2 Worked Example: Building the DFA for $(a \mid b)^*abb$

[DB] We continue directly from Lecture **??**'s 14-state NFA (§**??**): start state 7, single accept state 14, and the ε- and symbol-transitions built stage by stage in that lecture. Applying subset construction:

DFA State	Set of NFA States (after ε-closure)	On a	On b
A (start)	$\{7, 5, 3, 1, 8, 9\}$ – everything reachable from 7 with no input at all	B	C
B	ε-closure(move$(A, a)) = \{2, 10, 6, 11, 8, 5, 1, 3, 9\}$	B	D
C	ε-closure(move$(A, b)) = \{4, 6, 8, 9, 5, 3, 1\}$	B	C
D	ε-closure(move$(B, b)) = \{4, 12, 6, 13, 8, 5, 1, 3, 9\}$	B	E
E (accepting)	ε-closure(move$(D, b)) = \{4, 14, 6, 8, 5, 1, 3, 9\}$	B	C



Example: Reading This DFA in Plain English

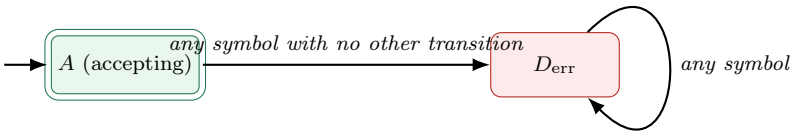
Only state E is accepting, and E is reached only by seeing an a then two b's in a row ($D \xrightarrow{b} E$, and D is reached only via $B \xrightarrow{b} D$, and B is reached only by having just read an a) – in other words, exactly "the last three characters read were `abb`." This is a direct, mechanical confirmation that the DFA really does recognize $(a \mid b)^*abb$: no hand-waving, just five states and the two helper operations above, run to completion.

Exam Focus

Notice the NFA had **14 states** but the DFA has only **5**. This is *not* a general rule – subset construction can just as easily *blow up* the state count (§6.3 works through why) – it simply happened to shrink here because many of this NFA's 14 states were ε-only "glue" states that collapse into the same reachable set. Do not assume the DFA is always smaller; always run the algorithm.

6.1.3 Dead States and the Implicit Error Transition

[DB] Step 2(b) of the algorithm above quietly skips any ε-closure(move(T, a)) that comes out empty. In a real scanner, "no valid transition on this symbol" cannot simply be ignored – it is exactly how a lexical error, or the end of the current token, gets detected. The standard way to make this explicit is to add one extra, non-accepting **dead state** D_{err} with a self-loop on every symbol, and route every missing transition to it.



Reaching D_{err} means the automaton is certain no continuation can ever lead back to an accepting state – so a real scanner treats arrival at D_{err} as the signal to stop extending the current match immediately, without even reading further input (there is no reason to; every path from D_{err} dead-ends). §6.4.2 shows exactly how this plugs into maximal-munch matching.

6.2 DFA Minimization

[DB] Subset construction guarantees a DFA, but not the *smallest* DFA for the language – different DFA states can behave identically in every possible future, making some of them redundant. **DFA minimization** (DB Algorithm 3.36) finds and merges exactly these redundant states, producing the unique smallest DFA (up to renaming states) recognizing the same language.

Definition: Distinguishable States

Two DFA states p and q are **distinguishable** if there exists *some* input string w such that following w from p ends in an accepting state while following w from q does not (or vice versa) – i.e. some future input reveals a difference in behavior between them. States that are *not* distinguishable by *any* string are **indistinguishable**, and can be safely merged into a single state without changing which strings the automaton accepts.

Algorithm 3.36 – Partition Refinement

Minimization works by repeatedly *refining* a partition of the states into groups that are *provisionally* treated as indistinguishable, splitting a group apart the moment a symbol is found that sends its members to genuinely different groups.

1. **Initial partition.** Split all states into exactly two groups: the accepting states F, and the non-accepting states $Q \setminus F$ (including the dead state, if one was added). This is the coarsest split that could possibly be correct, since an accepting and a non-accepting state are always distinguishable by the empty string $w = \varepsilon$ alone.
2. **Refine, repeatedly.** For each current group G and each input symbol a, check where each state in G goes on a. If every state in G lands in the *same* other group on every symbol, G stays together. If some states in G land in a *different* group than others (on some symbol a), split G into sub-groups accordingly – states now provably distinguishable (via that one symbol a) must never end up merged.
3. **Repeat until stable.** Keep refining until one full pass over every group and every symbol produces *no* further splits – at that point the partition has stabilized: every two states remaining in the same group really are indistinguishable by *any* string, not just the symbols checked so far.
4. **Build the minimized DFA.** Each final group becomes one state of the minimized DFA. The start state is the group containing the original start state; a group is accepting if it contains (any, and therefore every) original accepting state; transitions are inherited directly, since every state in a stable group agrees on where each symbol leads.
5. **Drop the dead state.** If minimization merges everything unreachable/hopeless into the dead-state group, that group is simply omitted from the final diagram – a missing transition in the minimized DFA is understood to mean "go to the (implicit) dead state and reject," exactly as §6.1.3 already described.

Exam Focus

The refinement in step 2 must be applied *simultaneously* across every symbol before deciding a group is stable – a group only survives a round if it agrees on *every* symbol in Σ, not just one. A common hand-execution error is stopping as soon as one symbol doesn't split a group, without checking the rest of Σ too.

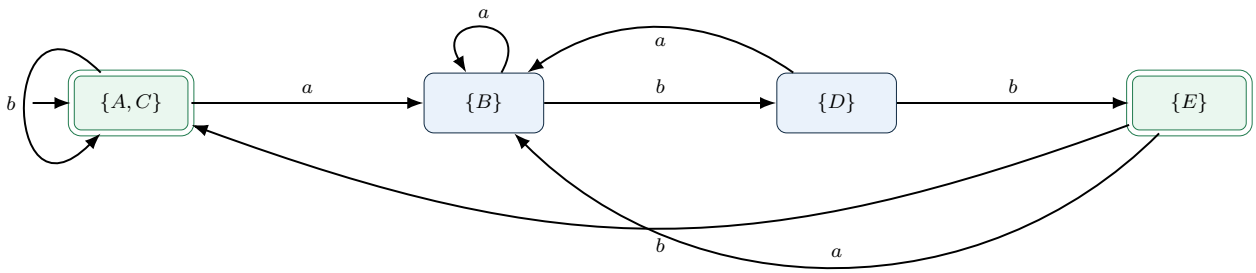
6.2.1 Worked Example: Minimizing the $(a \mid b)^*abb$ DFA

[DB] Apply the algorithm to the 5-state DFA built in §6.1.2 (A, B, C, D, E; only E accepting):

Round	Partition
Initial	$G_1 = \{A, B, C, D\}$ (non-accepting), $G_2 = \{E\}$ (accepting)
Round 1	Check G_1's members on each symbol. On a: $A \to B$, $B \to B$, $C \to B$, $D \to B$ – all land in G_1 (specifically all in B's group so far, fine). On b: $A \to C \in G_1$, $B \to D \in G_1$, $C \to C \in G_1$, but $D \to E \in G_2$ – **different group!** D must split off: G_1 becomes $G_{1a} = \{A, B, C\}$ and $G_{1b} = \{D\}$.
Round 2	Re-check $G_{1a} = \{A, B, C\}$: on a, all three go to $B \in G_{1a}$ – consistent. On b: $A \to C \in G_{1a}$, $B \to D \in G_{1b}$, $C \to C \in G_{1a}$ – B **disagrees!** Split again: G_{1a} becomes $\{A, C\}$ and $\{B\}$.
Round 3	Re-check $\{A, C\}$: on a, both go to B's group – consistent. On b: $A \to C$, $C \to C$ – both land in the *same* group $\{A, C\}$ itself – consistent. No further split needed for this group. All other groups are singletons, so they are trivially stable.
Stable	$\{A, C\}$, $\{B\}$, $\{D\}$, $\{E\}$ – four groups, no further splits possible.

Example: Why A and C Merge

Intuitively: A ("seen nothing useful yet, or just finished a full `abb` and are starting over") and C ("just saw a `b` that isn't continuing an `ab` prefix") behave *identically* from here on – from both, an a starts a fresh attempt at matching `abb`, and a b leaves you in exactly the same "waiting" situation. No future string can ever tell A and C apart, which is precisely the formal definition of indistinguishable states being applied.



The minimized DFA: 4 states instead of 5 – A and C have collapsed into one state, and every transition is inherited unchanged (e.g. $E \xrightarrow{b} C$ becomes $E \xrightarrow{b} \{A, C\}$). This is provably the smallest possible *DFA for $(a \mid b)^*abb$: DB Algorithm 3.36 guarantees minimality, not just a smaller result.*

Exam Focus

"Smallest possible" is a real, provable guarantee, not just "smaller than before." Two different minimized DFAs for the *same* language are always identical up to renaming states – there is exactly one minimal DFA (isomorphism aside), which is why minimization is sometimes described

as computing a *canonical form* for a regular language.

6.3 NFA vs. DFA, Revisited: Power and Performance

[DB] Lecture **??** and §**??** already established that NFAs and DFAs recognize *exactly* the same class of languages – the subset construction above is the constructive proof of that fact, run by hand. Equal recognizing power does *not* mean equal practical cost, though, and this distinction is worth making concrete now that you have built both an NFA and a DFA for the same language.

	NFA (simulated directly)	**DFA (pre-built, table-driven)**
States tracked per input symbol	A *set* of up to $\lvert Q_{\text{NFA}}\rvert$ states must be tracked and updated at every step (recomputing an ε-closure each time)	Exactly **one** state at all times – a single array index
Work per input symbol	$O(\lvert Q_{\text{NFA}}\rvert)$ or worse, since every state in the current set may need its own transitions and closure explored	$O(1)$ – one table lookup, `table[state][symbol]`
Number of automaton states	Exactly as many as Thompson's construction produced – linear in the size of the regular expression	Can be **exponentially larger** ($2^{\lvert Q_{\text{NFA}}\rvert}$ in the worst case) – see the classic blow-up example below
When the cost is paid	Every single run, on every input, pays the per-symbol simulation overhead	Paid **once**, offline, when the DFA (and its table) is built; every subsequent run is just table lookups

Example: The Classic Exponential Blow-up

Consider the regular expression $(a \mid b)^* a (a \mid b)^n$ for some fixed n – "the n-th-from-last character is an `a`." Its NFA (via Thompson's construction) has only $O(n)$ states. But its *minimal* DFA needs 2^n states: intuitively, the DFA must remember the *last n characters seen* (to know, once n more characters arrive, whether that remembered character really was n-from-last), and there are 2^n possible values for "the last n characters," each requiring a genuinely distinguishable state. This is a real, provable worst case, not a contrived edge case – it is exactly why some lexer generators offer a choice between DFA-based and NFA-based matching for pathological patterns.

Why Lexer Generators Overwhelmingly Choose the DFA Side

For *ordinary* lexical tokens (identifiers, numbers, keywords, operators), the exponential worst case above essentially never triggers in practice – real token patterns are small and well-behaved, so their DFAs stay modest in size. Given that, the DFA's $O(1)$-per-character, pre-paid-once cost is a clear win for a component like a lexer that runs on *every single character* of every source file, potentially billions of times across a large codebase. This is precisely why every mainstream lexer generator (Flex included) compiles down to a DFA rather than simulating an NFA at scan time, even though building that DFA (subset construction, then minimization) is itself extra

one-time work at generator-build time.

Exam Focus

Contrast this with §??'s discussion of backtracking regex engines (Perl, Python, JavaScript): those engines simulate something closer to an NFA *with backreferences* at match time, which is exactly why they can suffer ReDoS (§??) – unlike a lexer generator, they cannot always fall back to a pre-built DFA, because backreferences push the language being matched outside the regular languages entirely (§?? again). A lexer's patterns, by contrast, are always *genuinely* regular by construction, so the DFA route is always available.

Exam Focus

Lazy ("on-the-fly") DFA construction is the practical middle ground worth knowing by name: tools like Google's RE2 and many `grep` implementations build DFA states *on demand*, the first time they are actually reached during a real match, and cache them for reuse – getting the DFA's fast steady-state lookup without paying to build every theoretically-possible state up front. This matters most for patterns where the full DFA would be large but any single input only ever visits a small fraction of its states.

6.4 Designing a Lexical-Analyzer Generator

[DB] Everything so far in this lecture – and the two before it – has been about turning *one* regular expression into *one* DFA. A real lexer, though, must recognize *dozens* of different token patterns (`ID`, `NUM`, every keyword, every operator, ...) at once, decide which pattern produced the *longest* match at every position (maximal munch, §??), and hand the winning token off to the parser. This section builds that generic architecture – exactly what a lexer-generator tool constructs for you, and exactly what *any* such tool (Flex included) is doing under the hood, whatever its concrete input syntax looks like. *(The concrete `.l`-file syntax for Flex specifically is covered in the separate supplementary Flex notes, not here.)*

6.4.1 Combining Many Patterns Into One Automaton

[DB] A lexer-generator's input is fundamentally a list of **(pattern, action)** pairs – one regular expression per token, paired with the code that should run when that token is matched (typically: build the right `<token-name, attribute-value>` pair and return it). The combining step is a direct generalization of the union rule from Thompson's construction (§??):

1. Build (or imagine) a separate small NFA fragment $N(r_i)$ for each individual pattern $r_1, r_2, \ldots, r_k$, exactly as in Lecture ??.
2. Introduce one new overall start state, with an ε-transition to *every* fragment's own start state – precisely the union construction, generalized from two fragments to k.
3. **Critically, do *not* merge the fragments' accept states into one shared accept state** the way an ordinary two-way union would. Instead, each fragment $N(r_i)$ keeps its *own* accept state, tagged with *which* token r_i corresponds to.
4. Run subset construction (§6.1) on this combined NFA exactly as before. A resulting DFA state is accepting if its underlying set of NFA states contains *any* tagged accept state – and it is tagged with *that* token (or, if more than one tagged accept state appears in the same set, with whichever pattern is listed *first* in the original specification – see §6.4.3 below).

Example: A Miniature Combined Lexer: Two Patterns

Suppose a lexer needs only two token patterns: `KEYWORD_IF` for the literal string `if`, and `ID` for `letter(letter|digit)*`. Thompson's construction gives two small fragments; the combining step wires a shared start state into both, with each fragment's own accept state tagged `IF` or `ID` respectively. Subset construction then merges these into one DFA. Scanning the input `if5`: after `i`, then `f`, the DFA is in a state whose underlying NFA-state set includes *both* the `IF` fragment's accept state *and* the `ID` fragment's accept state simultaneously (since `if` is a valid lexeme for *both* patterns) – but reading one more character, `5`, only the `ID` fragment has anywhere left to go, so the DFA continues and eventually accepts `if5` as a single `ID` token, illustrating exactly why maximal munch (next subsection) has to keep reading past an early, shorter match.

6.4.2 Maximal Munch, Implemented as DFA Simulation

[DB] §?? introduced the longest-match rule informally. With a combined DFA in hand, it has a precise, mechanical implementation:

The Maximal-Munch Scanning Loop

1. Starting from the DFA's start state and the current input position, repeatedly follow transitions on successive input characters, **remembering the most recent point at which the current state was accepting** (and which token it was tagged with).
2. Keep advancing as long as a transition exists for the next character – *even past* an accepting state, since a longer match might still be possible (exactly the `if5` example above: the DFA is accepting for `IF` right after `if`, but a real scanner does not stop there).
3. Stop advancing the moment either (a) the input is exhausted, or (b) the next character has no valid transition from the current state – i.e. the only remaining move is to the implicit dead state D_{err} from §6.1.3. There is never a reason to enter D_{err} itself: once every remaining transition would lead there, the scan is over.
4. **Roll back** to the *last remembered accepting state* (step 1), emit a token for whichever pattern that state was tagged with, and resume scanning from the character right after that point.
5. If the current state was *never* accepting at any point during the scan (no remembered position exists at all), this is a genuine lexical error (§??): report it and invoke panic-mode recovery (§??).

Example: Tracing `<=` Against a Combined `RELOP` DFA

Suppose the DFA has transitions recognizing both `<` (alone, tagged `LT`) and `<=` (tagged `LE`). Scanning `<=10`: after `<`, the DFA is already in an accepting state (tagged `LT`) – remember this position. Reading `=` still has a valid transition (to the `LE`-tagged accepting state) – so keep going and update the remembered position to *here* instead (a strictly longer match was just found). Reading the next character, `1`, has *no* transition from the `LE` state – stop. Roll back to the last remembered position (right after `=`), emit `<LE, ->`, and resume scanning at `1`. This is exactly *why* `<=` is never mis-split into `<` followed by `=`, made completely mechanical.

6.4.3 Resolving Ties: What Happens When Two Patterns Match Equally Long

[DB] Maximal munch resolves *length* ties (always prefer the longer match), but a different kind of tie remains possible: two *different* patterns matching the *exact same* lexeme, of the exact same length. The reserved-word trick from §?? is one instance of this (`while` matches both a specific keyword pattern and the generic `ID` pattern) – and lexer generators resolve it with one simple, memorize-able rule:

Priority by Order of Declaration

When several patterns match a lexeme of the *same* maximal length, the lexer-generator convention is to prefer whichever pattern was **listed first** in the original specification. This is exactly why, in practice, keyword patterns like `if` and `while` are always written *before* the generic `ID` pattern in a lexer specification's rule list – listing them first guarantees the keyword-specific token wins the tie, without needing the reserved-word symbol-table trick from §**??** at all (both techniques solve the same problem; a lexer generator typically uses rule ordering, while a hand-written lexer more often uses the symbol-table trick).

Exam Focus

Rule order controls only *same-length* ties. It never overrides maximal munch itself: a *longer* match from a pattern listed *later* in the file still wins over a *shorter* match from a pattern listed earlier. Get the two straight: **length** beats **position in the file**; position in the file is only the tie-breaker when lengths are *equal*.

6.4.4 Input Buffering: Reading Characters Efficiently

[DB] Maximal munch (§6.4.2) has one uncomfortable-sounding consequence: the scanner sometimes reads several characters *past* the eventual end of a token before it knows to roll back (the `<=` example needed to peek at `1` before committing to `<=`). A naïve implementation that requests one character at a time from the operating system, and that can only look one step ahead, would need awkward, slow special-casing to support this "read ahead, then roll back" behavior. DB's classic fix is a purely mechanical buffering scheme that makes rollback essentially free.

Definition: Double Buffering with Sentinels

Maintain *two* buffers of equal, fixed size (each, e.g., one disk block), laid out contiguously in memory, plus two pointers:

- `lexemeBegin` marks the start of the lexeme currently being matched;
- `forward` scans ahead, one character at a time, exactly as maximal munch's simulation loop (§6.4.2) advances the DFA.

When `forward` reaches the end of one buffer, the *other* buffer is refilled (one system call, one full block) while scanning continues seamlessly across the boundary. Once a token is recognized, `lexemeBegin` jumps forward to just past it, and the cycle repeats. Rolling back `forward` to a remembered accepting position (§6.4.2) is therefore just moving a pointer *backward within already-buffered memory* – no re-read from disk, no special-casing at all.

A further trick avoids testing "have I run off the end of the buffer?" on *every single* character read (which would otherwise double the cost of the scanner's innermost loop): place a special **sentinel** character (conventionally `eof`, distinct from every real input character) at the very end of *each* buffer. The scanner's inner loop then needs only *one* check per character – "did I just read the sentinel?" – and only in that comparatively rare case does it do the extra work of figuring out *which* buffer's sentinel was hit (real end of file) versus a routine buffer-swap point.

Exam Focus

Be able to explain *why* double buffering (rather than a single buffer, or reading one character at a time from the OS) is necessary specifically because of maximal munch's "read-ahead-then-roll-back" behavior – this input-buffering material is frequently tested as a short conceptual

question ("why can't a lexer just read one character at a time from disk?") rather than as a diagram-drawing exercise. A closely related follow-up, made precise in §6.4.4.1 just below: the two buffers are *reused in rotation* as scanning proceeds through a file of any length, not permanently assigned to "the first half" and "the second half" of the source file.

6.4.4.1 The Two Buffers Are Recycled, Not Fixed Halves of the File

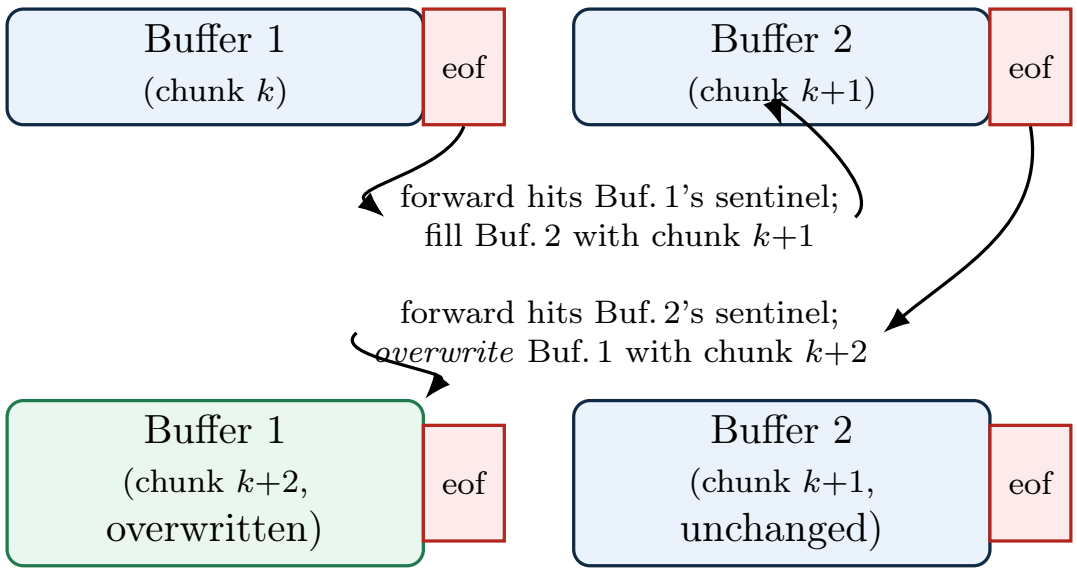
[DB] It is tempting to picture "Buffer 1" and "Buffer 2" as two permanent halves of the source file – the first half of the file lives in Buffer 1, the second half in Buffer 2, and that's the end of the story. That picture is wrong for any file longer than two blocks, and the correct picture is what actually makes double buffering useful for files of *any* length.

The Buffers Cycle: 1, 2, 1, 2, ...

Buffer 1 and Buffer 2 are just two fixed physical slots in memory. As `forward` marches through the file, the buffer it is *not* currently reading is the one that gets refilled with the *next* unread chunk of the file – regardless of whether that buffer's physical slot is labeled "1" or "2". Concretely:

1. `forward` scans through Buffer 1's current contents.
2. It hits Buffer 1's sentinel ⇒ Buffer 2 is (re)filled with the *next* chunk of the file, and `forward` continues into Buffer 2.
3. `forward` scans through Buffer 2's current contents.
4. It hits Buffer 2's sentinel ⇒ Buffer 1 is now *overwritten* with the chunk *after that* and `forward` continues into Buffer 1 again – physically the same memory slot as step 1, but holding entirely new characters.
5. Repeat: 1, 2, 1, 2, ... for as long as the file has more characters left.

So "the other buffer" at any moment just means "the one `forward` is not currently in" – it is always safe to overwrite, because `forward` has already moved past it and (as the safety condition below explains) so has `lexemeBegin`.



Top row: scanning chunk k in Buffer 1, chunk k+1 already waiting in Buffer 2. Bottom row: once `forward` has moved into Buffer 2, Buffer 1 is free and gets overwritten with chunk k+2 – the same physical slot*, new content. The cycle is 1, 2, 1, 2, ..., not "first half, second half."*

Definition: The One Safety Condition

A buffer may only be overwritten once *both* `forward` *and* `lexemeBegin` have moved past it – if `lexemeBegin` (marking a match still being rolled back to) were still sitting inside a buffer, overwriting it would destroy data a rollback still needs. In practice this is never a real constraint: buffers are a full disk block (e.g. 4 KB), and no realistic single lexeme spans anywhere near that much source text, so `lexemeBegin` is always still within the buffer `forward` currently occupies, or at worst the one just before it – never further back than that. (Lecture **??**'s treatment of retraction, §**??**, is the sharpest illustration of this: a retraction only ever steps `forward` back by *one* character, so it can never need to reach further back than the immediately preceding buffer.)

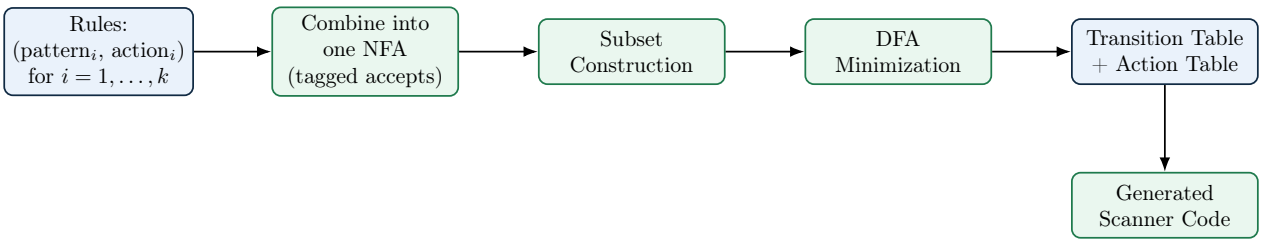
Example: Worked Trace: A Token Straddling the Buffer Boundary

To see *why* the cycle – and not just "two buffers exist" – is the point, trace scanning `x<=10;` with a deliberately tiny buffer size of **2 characters** (real buffers hold thousands; 2 is chosen purely so the boundary shows up almost immediately). Assume the DFA recognizes `RELOP` tokens `<` and `<=` (as in §6.4.2's earlier `<=` example) and `NUM` for digit sequences.

Step	Buffer contents	What happens
1	Buf. 1 = `x<|eof`	`forward` reads `x` (an `ID` on its own), then `<` – already an accepting `RELOP` (`LT`); remember this position.
2	Buf. 1 sentinel hit	Not true EOF, just Buf. 1's end – **fill Buf. 2** with the next chunk, `=1`. `forward` continues, now physically inside Buf. 2.
3	Buf. 2 = `=1|eof`	`forward` reads `=` – still a valid transition, to the `LE` accepting state (a *longer* match than `LT`); update the remembered position to *here*, right at the Buf. 1/Buf. 2 boundary.
4		`forward` reads `1` – no transition from `LE`. **Stop.** Roll `forward` back to the remembered position (right after `=`, i.e. Buf. 2 position 0). Emit `<LE, ->`. `lexemeBegin` jumps to Buf. 2 position 1 (the `1`).
5	Buf. 2 sentinel hit	`forward` re-reads `1` (start of a `NUM`), then hits Buf. 2's sentinel. Since `lexemeBegin` has already moved past all of Buf. 1's old content, **it is now safe to overwrite Buf. 1** with the next chunk, `0;`. `forward` continues, wrapping back into Buf. 1 – same physical slot as step 1, brand new characters.
6	Buf. 1 = `0;|eof` (new)	`forward` reads `0` – extends the `NUM` match to `10`. Reads `;` – no transition. Stop, roll back, emit `<NUM, 10>`.

Two things this trace makes concrete: first, the `<=` token's two characters came from *two different physical buffers* (`<` from Buf. 1, `=` from Buf. 2), and rolling `forward` back one position across that exact boundary was still just a pointer move – nothing was re-read from disk. Second, Buf. 1 gets reused (step 5→6) for entirely new characters (`0;`) once scanning has moved on – confirming the 1, 2, 1 cycle, not a fixed first-half/second-half split.

6.4.5 Putting It All Together: The Generic Lexer-Generator Pipeline



Everything left of "Generated Scanner Code" happens once, offline, when the lexer-generator tool runs on your rules file. What actually ships and runs on every source file afterward is only the rightmost box: a tight loop implementing §6.4.2's maximal-munch simulation over the pre-built table, driven by the double-buffered input of §6.4.4, and exposed to the parser via exactly the `getNextToken()` *interface §?? described. This is the complete answer to "what does a lexer-generator tool actually do" – Flex is one concrete implementation of this exact pipeline, and its own* `.l`*-file syntax for expressing the* Rules *box is covered separately.*

Closing the Loop Back to Lecture 3's Big Picture

Lay §??'s original roadmap diagram next to this section's pipeline and they match exactly, with one addition: Lecture ?? promised *regular expression* → *NFA* → *DFA* → *minimized DFA* → *transition table* → *lexer*, for *one* pattern. This lecture generalizes every arrow in that promise to *many* patterns at once (via the tagged-union combining step) and fills in the one detail the original diagram left implicit – how "transition table → lexer" actually happens, via maximal-munch simulation over double-buffered input. Nothing new was invented to make this jump; the entire generalization is the union rule from Thompson's construction, applied k times instead of twice. What is still missing – and what Lecture ?? picks up next – is the *other* way to reach a working scanner: writing one by hand, diagram by diagram, with no generator tool at all, and weighing that choice honestly against everything built here.

6.5 Self-Check Questions

Practice – Not Graded

1. Apply subset construction by hand to the NFA for ab^* (built in Lecture ??'s self-check question 5). List each DFA state as a set of NFA states, and draw the resulting DFA.
2. For the DFA you built in the previous question, apply DFA minimization. Are any states mergeable? Justify your answer in terms of distinguishability.
3. Explain, in your own words, why ε-closure must be applied *both* when computing the start state *and* again after every single `move`, not just once at the beginning.
4. Construct a small regular expression of your own (different from $(a \mid b)^*a(a \mid b)^n$) where you believe the DFA will have *more* states than the NFA, and explain your reasoning informally.
5. Trace the maximal-munch scanning loop (§6.4.2) on the input `intx` against a combined DFA for the two patterns `int` (keyword) and `letter(letter|digit)*` (`ID`), assuming keyword patterns are listed first. What single token is emitted, and why?
6. Two patterns, `a+` and `a+b?`, are both listed in a lexer specification, with `a+` listed first. For the input `aaa` (with no trailing `b` anywhere), which pattern's action fires, and is this a length tie or a maximal-munch decision?
7. Explain why a lexer cannot safely process input one character at a time, read directly from disk on every single character, given how maximal munch works – and explain concretely what a sentinel character saves the scanner's inner loop from having to do on every iteration.
8. A file is 10 KB long and each buffer holds 4 KB. In terms of the 1, 2, 1, 2, ... cycle from §6.4.4.1, which buffer holds the *final* chunk of the file, and how many times in total does each physical buffer get filled?

Key Takeaways

- **Subset construction** converts any NFA into an equivalent DFA by tracking *sets* of NFA states as single DFA states, using ε-closure and `move` as its two building blocks; termination is guaranteed because a finite NFA has only finitely many state subsets.
- A **dead state** makes the "no valid transition" case explicit, and is exactly the signal maximal-munch scanning uses to know a match cannot be extended any further.
- **DFA minimization** (partition refinement) merges states that no future input string could ever tell apart, producing the provably smallest DFA for a language – a canonical form, unique up to state renaming.
- NFAs and DFAs recognize *exactly* the same languages, but the practical trade-off is real: DFA simulation is $O(1)$ per character with a pre-paid one-time build cost, while NFA simulation pays a per-character cost proportional to the current state set on *every* run – and a DFA can, in the worst case, be **exponentially larger** than its source NFA.
- A lexer-generator tool builds *one* combined DFA out of *many* token patterns by generalizing Thompson's union rule to k patterns, tagging each pattern's own accept state; **maximal munch** is then just a DFA-simulation loop that remembers the last accepting position seen and rolls back to it; **same-length ties** between patterns are broken by whichever pattern was listed first.
- **Double buffering with sentinels** is what makes maximal munch's read-ahead practical: rollback becomes a cheap pointer move within already-buffered memory, and the sentinel collapses the scanner's per-character bounds check into a single, rare-case test. The two physical buffers are **reused in rotation** (1, 2, 1, 2, ...) as scanning proceeds through a file of any length – not permanently assigned to the file's first and second halves – and a buffer is only ever overwritten once both `forward` and `lexemeBegin` have moved past it.
- This lecture completes Lecture **??**'s original big-picture promise in full: regular expression → NFA → DFA → minimized DFA → transition table → working, table-driven, maximal-munch lexer – every arrow now built, worked by hand, and tied back to a concrete implementation architecture. Lecture **??** now looks at the same destination from the opposite direction: building a scanner by hand, without a generator, and comparing the two approaches honestly.

References for This Lecture

References

[DB] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Pearson, 2006.

[ET] Cooper, K. D. and Torczon, L. *Engineering a Compiler*. 2nd ed. Morgan Kaufmann, 2011.

[AP] Appel, A. W. *Modern Compiler Implementation in C/Java/ML*. Cambridge University Press, 2004.